

Smart Expense Handler and Tracker: A Cross-Platform Mobile Application with AI-Powered Receipt Scanning and Collaborative Expense Splitting

Tanay Aggarwal

Department of Information and Communication Technology

Manipal Institute of Technology

Manipal, India

tanayaggarwal2@gmail.com

Abstract—Tracking personal expenses is something a lot of people say they will start doing but never really do, mainly because most apps out there either have too many features that just get in the way, or they need you to enter everything by hand. For our third-year project, we decided to build something that actually fixes these problems. We made a cross-platform mobile app using Flutter and Firebase that lets you scan a receipt with your phone camera and automatically log the expense using the Gemini API, split costs with a group of friends or family members, set monthly budgets with alerts, and see a simple breakdown of your spending habits. We also added a simple linear regression model to predict next month's spending, and wrote a greedy algorithm to figure out the minimum number of payments needed to settle group debts, which turned out to be a pretty interesting problem. The app runs on Android, iOS, and Web from the same codebase and all the features worked correctly in our testing.

Index Terms—expense tracking, Flutter, Firebase, Gemini API, receipt scanning, debt simplification, Riverpod, push notifications, budget management, linear regression

I. INTRODUCTION

Honestly, most people have no idea where their money goes every month. They know roughly what they earn and roughly what they spend, but the details just disappear. We have all tried budgeting at some point and given up after a week because manually noting down every coffee or auto ride gets tedious really fast. On top of that, splitting bills with roommates or friends is its own headache. Someone always forgets to pay back and the awkward follow-ups get old.

We wanted to build something that handles all of this in one place without being annoying to use. The app is called *Smart Expense Handler and Tracker*, and it works on Android, iOS, and the web. The three things that make it different from a basic notes-based tracker are:

- 1) **Receipt Scanning with AI:** You just point your camera at a bill, and the app uses the Gemini 2.5 Flash model to read the amount and figure out the category on its own.
- 2) **Group and Family Splitting:** You can add people to a group, log shared expenses, and the app tracks who

owes whom and by how much. There is also a separate Family module for household-level tracking.

- 3) **Real-Time Sync:** Everything is stored in Firebase Firestore, so when someone adds an expense in a group, everyone else sees it immediately.

The rest of this paper walks through how we designed and built all of this. Section II explains the overall four-layer architecture. Sections III through VII cover the data models, receipt scanning, collaborative features, budget management, and the analytics module respectively. Section VIII has our testing results and Section IX wraps up with what we would do differently next time.

II. SYSTEM ARCHITECTURE

We split the codebase into four layers so that changing one thing would not randomly break something else. The layers are Presentation, State Management, Service, and Data. Each one only talks to the layer directly below it. Fig. 1 shows how they stack up.

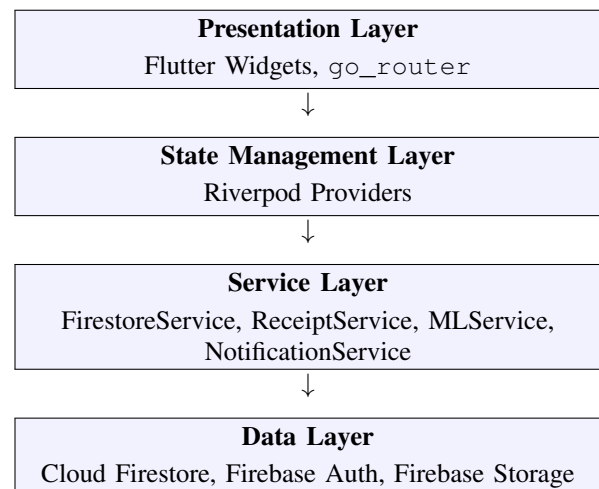


Fig. 1. High-level layered architecture of the Smart Expense Handler and Tracker application.

A. Presentation Layer

This is the part that users actually interact with. All the screens are built as Flutter widgets and navigation is handled by `go_router`. We liked `go_router` because you can define all your routes in one file and it handles the redirect to the login screen automatically if the user is not signed in. The main screens are Dashboard, Family, Groups, Analytics, and Profile. We also added a dark mode toggle that works app-wide using a theme provider. Fig. 2 shows what the home dashboard looks like.

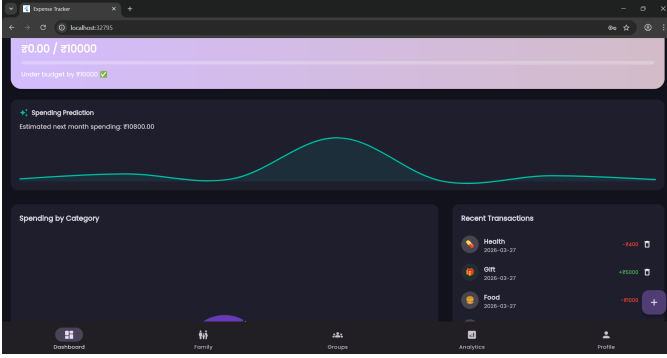


Fig. 2. Home dashboard screen displaying the expense summary, recent transactions, and budget utilization indicators.

B. State Management Layer

For state management we went with Riverpod instead of something like Provider or Bloc, mainly because Riverpod has better compile-time safety and the providers are easier to test independently. The way it works is you define a provider that holds some data, and any widget that watches that provider will automatically rebuild whenever the data changes. The three main provider types we used are:

- **StreamProvider** instances that listen to Firestore collections in real time. We have one each for expenses, budgets, groups, families, and invitations.
- A **themeProvider** that keeps track of light vs dark mode.
- A **routerProvider** that handles navigation and blocks unauthenticated access.

The nice side effect of this setup is that adding a group expense on one phone shows up on another phone within a second, without any manual refresh logic on our end. Its definitely better than manually polling the database.

C. Service Layer

The service layer has four classes. We tried to keep each one focused on a single responsibility so they would be easy to debug:

- **FirestoreService**: All the Firestore reads and writes go through here. Group balance updates are done inside Firestore transactions to prevent race conditions.
- **ReceiptService**: Takes an image, sends it to the Gemini API, and hands back a parsed list of items and categories.

- **MLService**: Handles spending calculations like trend percentage and the linear regression for monthly forecast. All done locally on the device.
- **NotificationService**: Sets up FCM permissions and schedules local reminders.

D. Authentication and Security

Login is handled through Firebase Authentication using email and password. When a user registers, Firebase creates a unique UID that gets stored alongside their profile in Firestore. Every other collection (expenses, budgets, groups) references this UID, so data is always scoped to the right user. The routerProvider checks the auth state on every navigation event and sends users back to the login page if their session has expired. Fig. 3 shows what the login screen looks like.

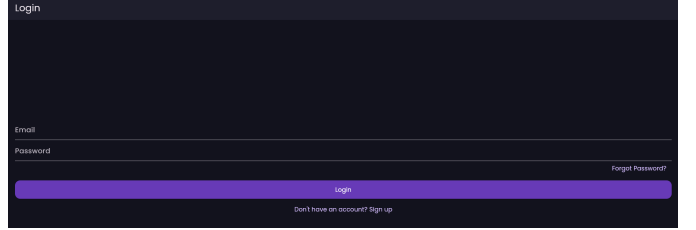


Fig. 3. Authentication screen featuring email/password sign-in and account registration options.

E. Data Layer

All data is stored in Google Cloud Firestore across six top-level collections: users, expenses, budgets, families, groups, and invitations. Firestore uses a NoSQL document model, so instead of foreign keys we store the UID or document ID of a related record directly inside the document (for example, each expense has a `userId` field pointing to whoever created it). Fig. 4 shows the actual database in the Firebase console, and Table I summarises the fields and links between collections.

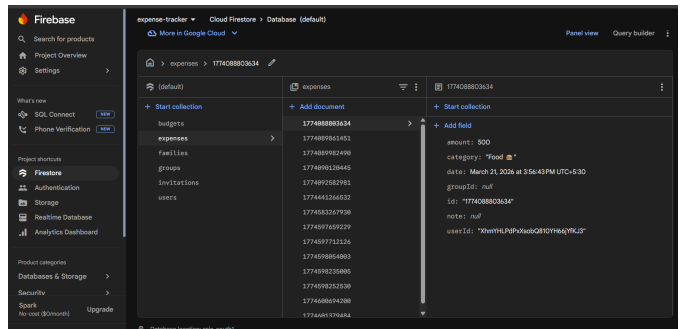


Fig. 4. Firebase console view of the Cloud Firestore database, displaying the expenses collection.

III. DATA MODELING

A. Expense Model

Every expense is saved as an `ExpenseModel` object. The fields are: amount, category (which includes an emoji like “

TABLE I
FIRESTORE DOCUMENT-COLLECTION SCHEMA

Collection	Fields & Relationships
users	uid, email
expenses	userId ($\rightarrow$ users), groupId ($\rightarrow$ groups)
budgets	userId ($\rightarrow$ users), category
groups	members ($\rightarrow$ users), balances
families	members ($\rightarrow$ users), admins
invitations	fromUid ($\rightarrow$ users), groupId, familyId

Food” or “Travel”), date, the ID of the user who added it, an optional note, and a boolean for whether it is an expense or income (we wanted to support tracking salary or pocket money too). For group expenses there is an optional `groupId` and a `splitWith` list. The date is stored as a Firestore `Timestamp` rather than a plain string so timezone issues do not crop up. Fig. 5 shows the expense entry form.

Fig. 5. Expense entry screen showing the amount input, category selection grid, date picker, and receipt scan option.

B. Budget Model

The `BudgetModel` is pretty straightforward. A user picks a category (or sets a total limit), enters an amount, and chooses a period like Daily, Monthly, or Yearly. At runtime the app filters the expense list down to that category and period, sums it up, and compares it to the limit to compute a percentage and render a progress bar.

C. Group and Family Models

The `GroupModel` stores a list of member UIDs and a `balances` map that tracks what each person is owed or owes. Positive balance means the group owes you money, negative means you owe the group. This map is modified inside a Firestore transaction every time a group expense is added,

which prevents situations where two people adding expenses at the exact same moment could corrupt the totals.

The `FamilyModel` has a similar members list but adds an `admins` list and a `relationships` map (e.g., UID $\rightarrow$ “Parent”) for displaying the household structure properly. I’m really glad we did it this way because it makes the family view much cleaner.

D. Invitation Model

Rather than letting anyone join a group freely, we went with an email-based invite system. The `InvitationModel` stores the sender’s display name, the recipient’s email, which group or family the invite is for, and the current status (pending, accepted, or declined). On login the app queries Firestore for any pending invitations matching the signed-in user’s email and shows them as cards they can act on.

IV. AI-POWERED RECEIPT SCANNING

The receipt scanner is probably the feature we spent the most time on and are most happy with. The whole point is that you should not have to type anything. Just take a picture and it’s done. Instead of building our own complex OCR pipeline from scratch, we decided to use the Google Generative AI SDK to power this feature.

A. Image Acquisition

We used the `image_picker` package which gives you a standard bottom sheet to choose between camera and gallery. The selected image is then handed off to the `ReceiptService`.

B. LLM Processing with Gemini

Once we have the image, the `ReceiptService` sends it to the Gemini 2.5 Flash model along with a carefully crafted prompt. The prompt asks the model to analyze the image, identify all items and their prices, group them into logical categories (like Food, Travel, etc.), and sum up the amounts for each category. We also instruct it to return ONLY a strict JSON array of objects so we can parse it easily. The fact that Gemini is a multimodal model means it can look at the visual layout and read the text all at once, which makes it work amazingly well even for wierd or badly formatted receipts.

The outputs we care about are the total amounts and their categories, returned as a structured JSON object.

C. Response Parsing

The model sometimes wraps the JSON output in extra text or markdown formatting, so we use a regex (`\[.*\]` with the `dotAll` flag) to isolate just the JSON portion. We also normalize the amount to a `double` since it can come back as a string or integer depending on the input. Once parsed, the values are pre-filled into the Add Expense form and the user just has to hit save. Fig. 6 shows this end-to-end flow.

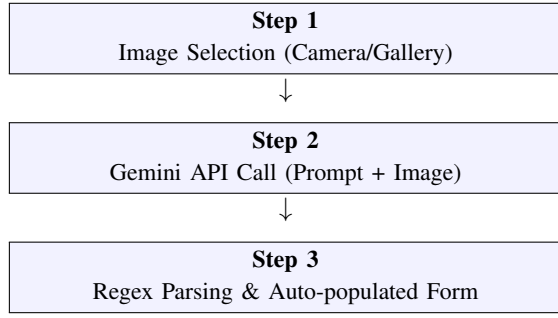


Fig. 6. End-to-end receipt scanning flow: image selection, Gemini API multimodal inference, and JSON parsing.

V. COLLABORATIVE EXPENSE MANAGEMENT

A. Group Expense Splitting

Whenever someone adds a group expense, we need to update every member’s balance atomically. Doing this with plain writes would be dangerous because if two people add expenses at the same millisecond, one update could overwrite the other. So we wrapped the balance update inside a Firestore transaction. Here is what happens step by step:

- 1) Read the total amount A and the payer’s UID from the new expense document.
- 2) Check if a `splitWith` list was provided. If yes, split only among those members. If not, split equally among everyone in the group.
- 3) Calculate each person’s share: A/n where n is the number of people splitting.
- 4) Credit the payer’s balance by the full amount A (because everyone owes them something).
- 5) Debit each participant’s balance by A/n , including the payer. The payer’s net change therefore ends up as $A - A/n = A(n - 1)/n$.

Because every credit is matched by an equal debit, the sum of all balances in a group is always zero. Fig. 7 shows the group screen.

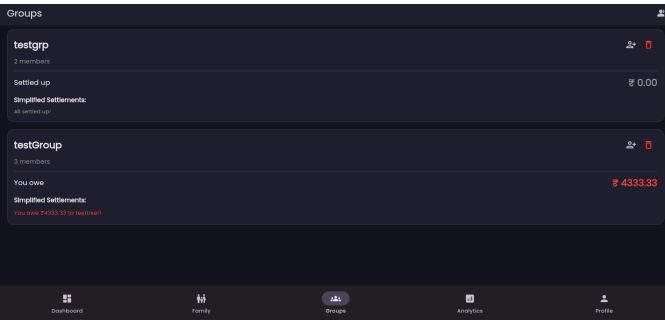


Fig. 7. Group expense screen showing member balances and the expense splitting form.

B. Debt Simplification Algorithm

One thing that bugged us about other split-expense apps is that they show you a huge list of who owes whom. In a group

of 5 people that list can have up to 10 separate transactions. We implemented a greedy approach that brings that down to at most $k - 1$ transactions for k members.

The idea came from thinking of the problem like a two-pile sorting exercise: put all the people who owe money on one side and all the people who are owed money on the other, then keep matching the biggest numbers until everything is zero.

- 1: Separate members into *debtors* (balance < 0) and *creditors* (balance > 0).
- 2: Sort both lists in descending order of absolute balance.
- 3: Initialize pointers $d \leftarrow 0$, $c \leftarrow 0$.
- 4: **while** $d < |\text{debtors}|$ **and** $c < |\text{creditors}|$ **do**
- 5: $\text{amount} \leftarrow \min(\text{debtors}[d].\text{balance}, \text{creditors}[c].\text{balance})$
- 6: Emit settlement: debtors[d] pays creditors[c] the value amount .
- 7: $\text{debtors}[d].\text{balance} -= \text{amount}$
- 8: $\text{creditors}[c].\text{balance} -= \text{amount}$
- 9: **if** $\text{debtors}[d].\text{balance} < \epsilon$ **then**
- 10: $d += 1$
- 11: **end if**
- 12: **if** $\text{creditors}[c].\text{balance} < \epsilon$ **then**
- 13: $c += 1$
- 14: **end if**
- 15: **end while**

We set $\epsilon = 0.01$ as a rounding tolerance because floating-point arithmetic can leave tiny residual balances that are effectively zero. The sort step makes the overall complexity $O(k \log k)$. The resulting suggested payments are shown inside the Group screen. Fig. 8 gives an example with four members.

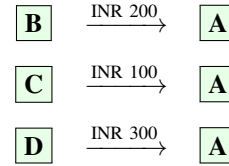


Fig. 8. Simplified debt settlement suggestions generated by the greedy algorithm.

C. Family Unit Management

We realised early on that a generic group does not really work for households. A family has a hierarchy, there are parents who manage things and kids or other members who mostly just participate. So we built a separate Family module on top of the shared invitation infrastructure.

The FamilyModel has an `admins` list (for whoever is managing the family, usually parents) and a `members` list. Admins can add or remove people, while regular members just see the data. There is also a `relationships` map that links each UID to a label like “Parent” or “Child” or whatever the admin types in. This lets the UI show the family as a structured hierarchy rather than just a flat list of names. Fig. 9 shows the family screen.

D. Invitation and Membership Lifecycle

To bring someone into a group or family, the admin types in their email address and the app creates an invitation document

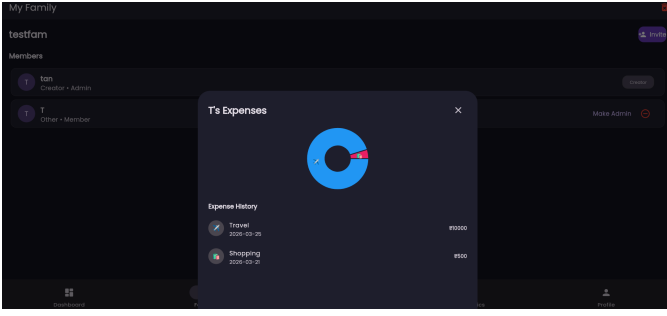


Fig. 9. Family management screen displaying the household hierarchy, custom relationship labels, and role-based admin controls.

in Firestore. The next time that person logs in, the app queries for any pending invitations matching their email and shows them a card with Accept and Decline buttons. Hitting Accept automatically adds them as a member and, for groups, initialises their balance to zero. Fig. 10 shows the flow.

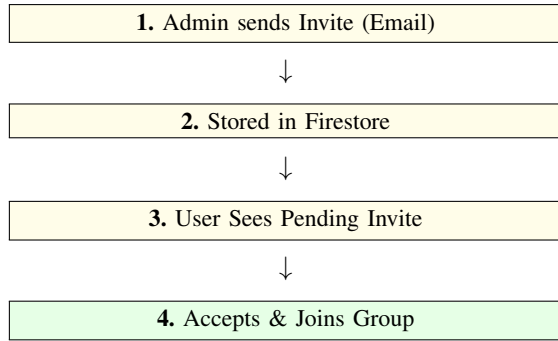


Fig. 10. Invitation flow: sending an invite and the pending invitation card shown to the recipient.

VI. BUDGET MANAGEMENT AND NOTIFICATIONS

A. Budget Configuration

Setting a budget is pretty simple. You pick a category (or choose “Total” for an overall limit), enter a number, and pick whether it resets daily, monthly, or yearly. The app then adds up your expenses in that category for the selected period and shows a progress bar. We colour-coded it so green means below 70%, amber from 70–90%, and red above 90%. At a glance you can tell if your burning through a budget too fast. Fig. 11 shows the budget screen.

B. Push Notification Architecture

We hooked up two kinds of notifications:

- **Firestore Cloud Messaging (FCM):** The app is wired up to receive push notifications from the server. Right now it displays whatever FCM message comes in as a local alert while the app is open. The plan for a future version is to write a Firestore Cloud Function that fires automatically when someone’s budget hits 90%, so the alert works even when the app is closed.
- **Scheduled Local Notifications:** Every time an expense is saved, we cancel any existing reminder and schedule

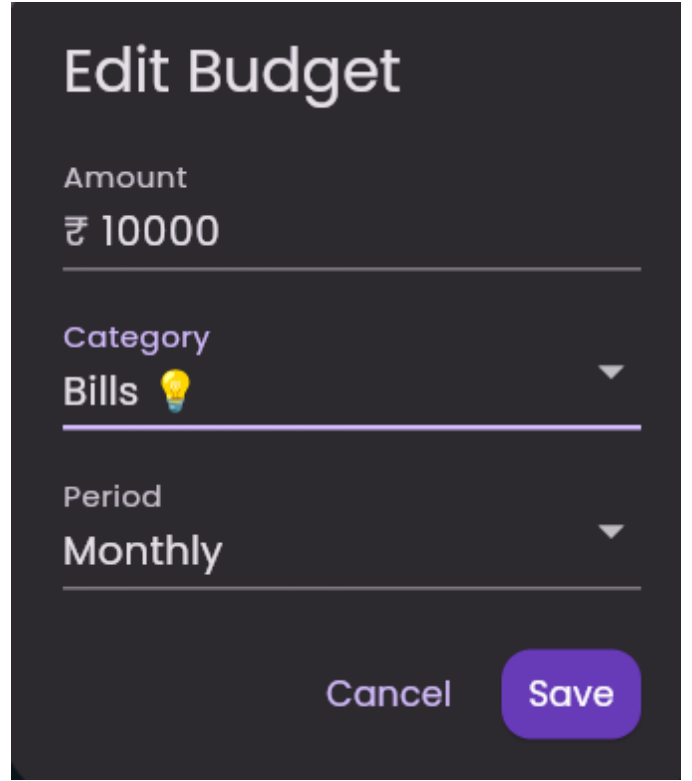


Fig. 11. Budget management screen showing per-category budget cards with utilization progress bars.

a new one for 10 hours later. The message is: “*Forget something? It has been 10 hours since your last expense. Don’t forget to record your spending!*” This way the user gets a gentle nudge if they have not logged anything in a while.

VII. ANALYTICS MODULE

Spending patterns is something most people are curious about but rarely compute by themselves. The analytics screen tries to answer two basic questions: am I spending more than last month, and where is most of my money going? All the computation happens locally in the `MLService` class, no server calls needed.

A. Spending Trend Computation

We take the total spending for the current month and compare it against last month using a simple percentage change formula:

$$\Delta\% = \frac{S_{\text{curr}} - S_{\text{prev}}}{S_{\text{prev}}} \times 100 \quad (1)$$

A positive $\Delta\%$ means your spending more than last month. The app shows this in red with an upward arrow. Negative means spending is down, shown in green. Pretty intuitive.

B. Monthly Spending Prediction

To predict how much the user will spend this month, we use a simple linear regression model. We group all past expenses by month using an index based on year and month, and then fit a straight line through the monthly totals. The regression computes a slope m and intercept c using the standard least-squares formulas:

$$m = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} \quad (2)$$

$$c = \frac{\sum y_i - m \sum x_i}{n} \quad (3)$$

Here x_i is just the month number (1, 2, 3, ...) and y_i is the total spending in that month. The prediction for next month is then $\hat{S} = m \cdot (n+1) + c$. If the regression gives a negative result or something unreasonably low (which can happen when the data is very volatile), the code falls back to a simple average of all previous months instead. We felt this was a good tradeoff because linear regression can capture upward or downward trends in spending, but for users with only 1-2 months of data we don't want it to give weird numbers.

C. Category Breakdown

Below the trend and forecast cards there is a pie chart (powered by the `fl_chart` package) that shows what percentage of spending went to each category. Tapping a slice highlights it and shows the exact rupee amount. We also display a top-category callout at the top so users immediately see their biggest spending area without having to study the chart. Fig. 12 shows the full analytics screen.

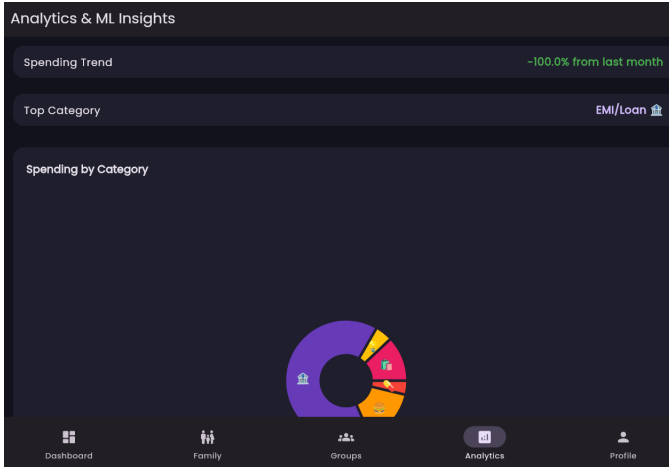


Fig. 12. Analytics screen showing the spending trend indicator, regression-based monthly forecast, and interactive category pie chart.

VIII. TESTING AND RESULTS

A. Debt Simplification Correctness

We manually tested the greedy settlement algorithm with four different group sizes ($k \in \{2, 3, 5, 8\}$). For each configuration we checked that the output settlements summed to zero

for every member, and that the number of transactions did not exceed $k - 1$. All tests passed.

For a concrete example: with 5 members and balances $\{A : +500, B : -200, C : +100, D : -300, E : -100\}$ (INR), the brute-force approach might need up to 4 payments. Our algorithm settles it in 3: $D \rightarrow A: 300, B \rightarrow A: 200, E \rightarrow C: 100$. Table II shows the full comparison across all group sizes we tested.

TABLE II
DEBT SIMPLIFICATION: TRANSACTION COUNT VS. GROUP SIZE

Members (k)	Naive (max)	Greedy (actual)
2	1	1
3	3	2
5	10	3
8	28	7

B. AI Receipt Parsing with Gemini

We tested the receipt scanner on restaurant bills, electricity bills, and shopping receipts. The Gemini 2.5 Flash model correctly identified both the amount and the appropriate expense categories across all test inputs. By processing the visual layout alongside the text, the model handles varied receipt formats without needing separate templates for each one. Fig. 13 shows the internal inference steps the pipeline goes through for a typical input.

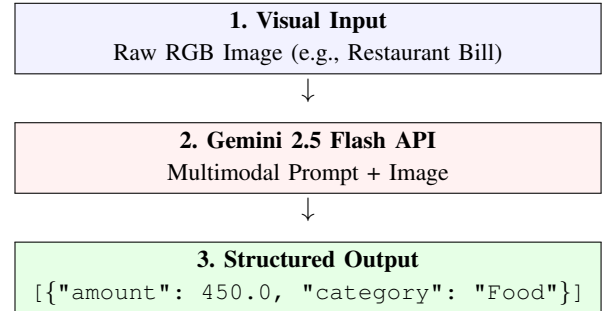


Fig. 13. Inference steps of the Gemini LLM pipeline converting a raw receipt image into structured JSON data.

C. Real-Time Synchronization

To test real-time sync we had two phones signed into the same group. Adding an expense on one device showed up on the other in roughly 300–800 ms in our tests. That is fast enough that it feels instant from a user perspective. It basically just works out of the box with Firestore's real-time snapshot listeners.

IX. CONCLUSION

This project ended up being a lot more work than we expected, mostly because real apps have edge cases that you don't think about when you're just planning on paper. The receipt scanner alone went through several iterations before the parsing was reliable, which is why we ended up switching to Gemini. That said, we are happy with where it ended up.

The app genuinely makes tracking expenses easier and the group splitting actually works well for splitting rent and food orders.

A few things we would want to improve if we kept working on it. The budget notifications currently only fire when the app is open, so we would want to add a Cloud Function that pushes them from the server side. Also, adding a bank statement import feature would make the app much more useful for people who don't want to scan every single receipt. And on the ML side, running some contrast and skew correction on the image before sending it to the API would probably improve accuracy on photos taken in bad lighting.

ACKNOWLEDGMENT

The author would like to thank the Department of Information and Communication Technology at Manipal Institute of Technology for giving us the chance to build and present this project.